

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Paralelne i distribuirane arhitekture i jezici

Strukture i enumeracije

Branislav Ristić

Zimski semestar, školska 2026./27.

Strukture

Strukture

- Strukture imaju više polja, povezanih u celinu.
- Slično torkama, ali sa imenovanim poljima.
- Polja mogu biti različiti tipovi podataka.
- Jasnije i fleksibilnije od torki.
- Ključna reč `struct`.
- Pristup putem tačka operatora (`.`).
- Sintaksa `..instanca` preuzima preostala polja iz druge instance (premešta ili kopira, zavisno od tipa polja).
- Mutabilnost na nivou strukture:
  - Nije moguće postaviti određena polja kao nepromenljiva.

Strukture: izvođenje osobina (`derive`)

- Atribut `#[derive(...)]` traži od kompajlera da implementira osobinu (*trait*) za tip, sa podrazumevanim ponašanjem.
- `Debug`: ispis sa `{:?}`, a `{:#?}` daje čitljiviji ispis u više redova.
- `Clone`: eksplicitna kopija metodom `clone()`; sva polja moraju biti `Clone`.
- `Copy`: implicitno kopiranje; sva polja moraju biti `Copy`, a tip mora biti i `Clone`.
- `PartialEq`: poređenje operatorima `==` i `!=`.
- `Display` (`{}`) se ne može izvesti; implementira se ručno.
- Osobine su tema sledećeg predavanja.

Strukture: primer

`primeri/structs-enums/src/bin/structure.rs`

```rust
// We derive the Debug trait for User struct
#[derive(Debug)]
struct User {
    active: bool,
    username: String,
    email: String,
    sign_in_count: u64,
}

// Implement the Display trait for User because it is not derivable
impl std::fmt::Display for User {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(
            f,
            "User: {} {} {} {}",
            self.active, self.username, self.email, self.sign_in_count
        )
    }
}

// Debug trait for User has the default implementation (derive)
// impl std::fmt::Debug for User {
//     fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
//         // In this regular string:
//             // Each " inside the string must be escaped as \".
//             // Each \ must be escaped as \\. This is because \ is an
 ↪  escape character in Rust strings.
```

```
//         write!(f, r#"{{"active": {}, "username": "{}", "email": "{}",
 ↪  "sign_in_count": {}}}"#, self.active, self.username, self.email,
 ↪  self.sign_in_count)
//     }
// }

// Implementing methods for User struct
impl User {
    // Constructor
    fn build_user(email: String, username: String) -> User {
        User {
            active: true,
            username,
            email,
            sign_in_count: 1,
        }
    }

    // Getter method
    fn get_email(&self) -> &String {
        &self.email
    }

    // Setter method
    fn set_email(&mut self, email: String) {
        self.email = email;
    }
}

fn main() {
    // Slide 3
    let user1 = User {
        email: String::from("someone@example.com"),
        username: String::from("someusername123"),
        active: true,
        sign_in_count: 1,
    };

    // Display and Debug
    println!("User : {}", user1);
    println!("User : {:?}", user1);

    // Using the constructor
    let user1 = User::build_user(
        String::from("someone@example.com"),
        String::from("someusername123"),
    );
    println!("User : {}", user1);
    println!("User : {:?}", user1);

    // Slide 4
    let user2 = User {
        email: String::from("another@example.com"),
        ..user1
    };

    // Display both user1 and user2
    // println!("User : {}", user1); // Error: `user1` is partially moved
```

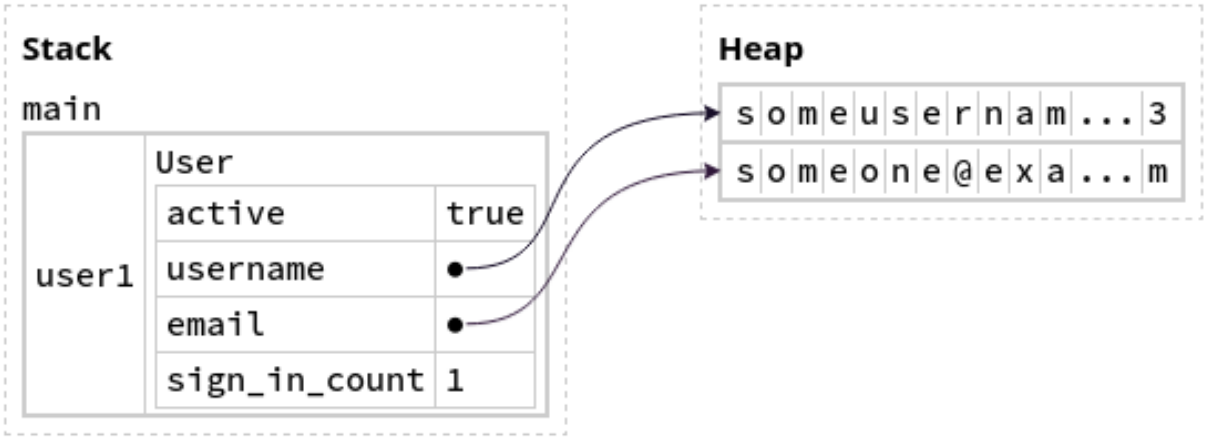
```
    println!("User : {}", user2);

    let mut user3 = User::build_user(
        String::from("third@example.com"),
        String::from("thirdusername123"),
    );
    user3.set_email(String::from("fourth@example.com"));
    println!("User email : {}", user3.get_email());
}
```

Izlaz

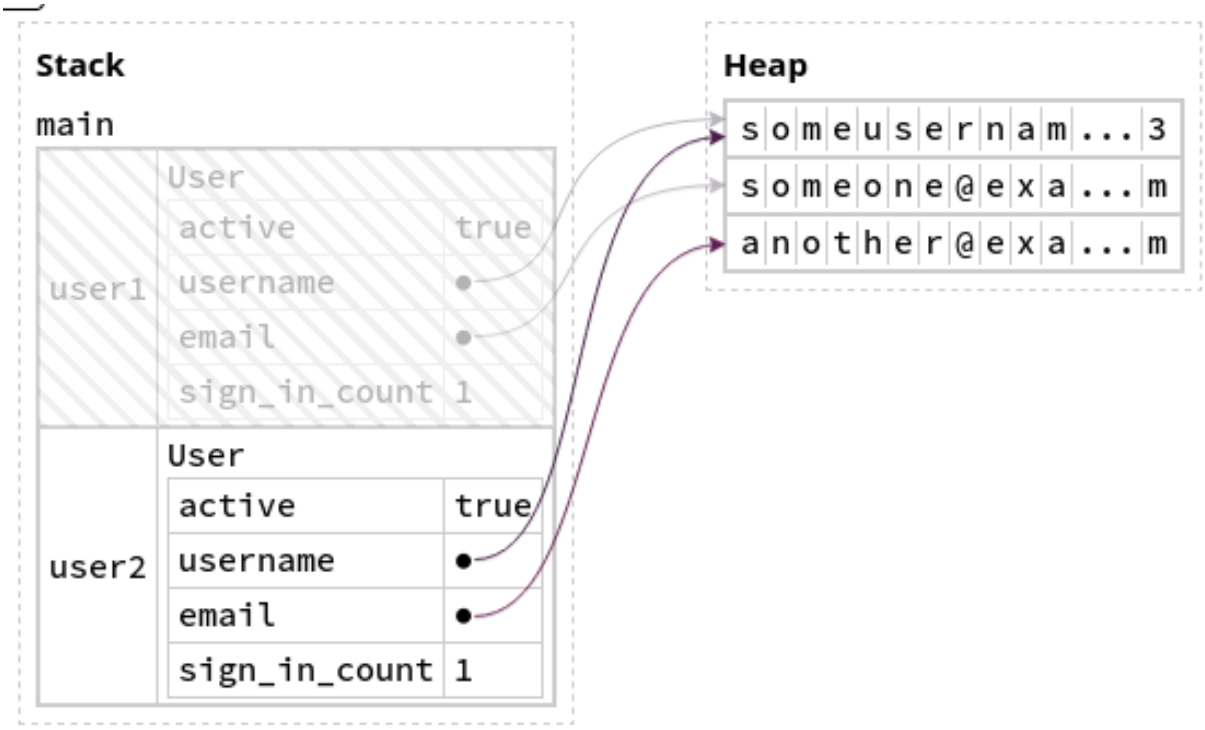
```
User : User: true someusername123 someone@example.com 1
User : User { active: true, username: "someusername123", email:
 ↪  "someone@example.com", sign_in_count: 1 }
User : User: true someusername123 someone@example.com 1
User : User { active: true, username: "someusername123", email:
 ↪  "someone@example.com", sign_in_count: 1 }
User : User: true someusername123 another@example.com 1
User email : fourth@example.com
```

Strukture: User u memoriji



Slika 1: Struktura User u memoriji

Strukture: user2 preuzima polja iz user1



Slika 2: Posle kreiranja user2: username je premešten iz user1, email je nov

Struktura torke

- Struktura torke omogućava grupisanje vrednosti bez imenovanja polja.
- Kreira se zaseban tip, za razliku od standardnih torki.
- Pristup preko indeksa:
  - Kao i kod torki.
- Korisno za razlikovanje sličnih podataka.

Struktura torke: primer

primeri/structs-enums/src/bin/tuple_structure.rs

```rust
// This structure represents a color in the RGB color space.
struct Color(i32, i32, i32);
// This structure represents a point in the 3D space.
struct Point(i32, i32, i32);

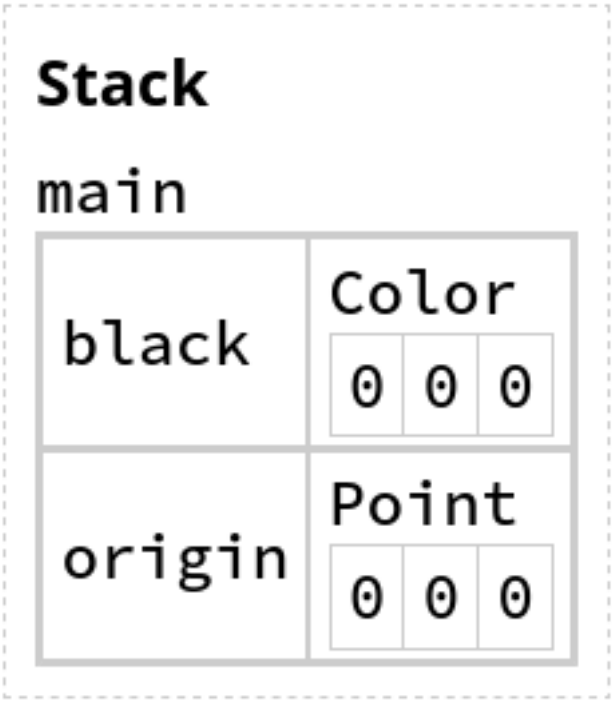
fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0, 0, 0);

    // Fields are accessed by index, as in tuples
    println!("black: ({}, {}, {})", black.0, black.1, black.2);
    println!("origin: ({}, {}, {})", origin.0, origin.1, origin.2);
}
```

Izlaz

```
black: (0, 0, 0)
origin: (0, 0, 0)
```

Struktura torke: Color i Point u memoriji



Slika 3: Color i Point na stack-u

Jediničnolika struktura bez polja

- Struktura bez polja, slično ().
- Korisno za nešto što ne zahteva podatke.
- Instance dele tip ali ne sadrže vrednosti.

Jediničnolika struktura bez polja: primer

primeri/structs-enums/src/bin/unit_structure.rs

```
// Automatically implement the Debug trait for AlwaysEqual, allowing it to
 ↪  be printed with {:?}
#[derive(Debug, Eq)]
struct AlwaysEqual;

// Implement the PartialEq trait for AlwaysEqual, allowing comparison with
 ↪  `==`
// Purpose of PartialEq: The PartialEq trait allows us to define custom
 ↪  equality logic for types,
// meaning we can decide what it means for two values to be "equal" (==).
// Eq is not automatic: requested above with `derive(Eq)`
impl PartialEq for AlwaysEqual {
    // Define the `eq` method to always return true, meaning any instance
     ↪  of AlwaysEqual
    // will always be considered equal to any other instance of
     ↪  AlwaysEqual
    fn eq(&self, _: &Self) -> bool {
        true
    }
}
```

```rust
// Eq has no methods, but requires PartialEq
// Eq promises: eq is reflexive, symmetric and transitive
// The compiler cannot check it: the programmer is responsible
// Eq (with Hash) is needed for HashMap keys and HashSet elements

fn main() {
    let subject1 = AlwaysEqual; // Create an instance of AlwaysEqual
    let subject2 = AlwaysEqual; // Create another instance of AlwaysEqual

    // Use the `==` operator, which calls the `eq` method, which always
     ↪  returns true
    if subject1 == subject2 {
        // Since `eq` always returns true, this branch will always execute
        println!("subject1 is always equal to subject2");
    } else {
        // This branch will never execute because `eq` is hardcoded to
         ↪  return true
        println!("This will never print");
    }

    // Use the function to compare the two instances
    if subject1.eq(&subject2) {
        // This branch will always execute because `eq` is hardcoded to
         ↪  return true
        println!("subject1 is always equal to subject2");
    } else {
        // This branch will never execute because `eq` is hardcoded to
         ↪  return true
        println!("This will never print");
    }

    if subject1 != subject2 {
        // This branch will never execute because `ne` is hardcoded to
         ↪  return false
        println!("This will never print");
    } else {
        // Since `ne` always returns false, this branch will always
         ↪  execute
        println!("subject1 is always equal to subject2");
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_reflexivity() {
        let a = AlwaysEqual;
        assert_eq!(a, a); // Reflexivity: a == a should be true
    }

    #[test]
    fn test_symmetry() {
        let a = AlwaysEqual;
        let b = AlwaysEqual;
        assert_eq!(a, b); // Symmetry: a == b
```

```
        assert_eq!(b, a); // b == a should also be true
    }

    #[test]
    fn test_transitivity() {
        let a = AlwaysEqual;
        let b = AlwaysEqual;
        let c = AlwaysEqual;
        assert_eq!(a, b); // a == b
        assert_eq!(b, c); // b == c
        assert_eq!(a, c); // a == c (transitivity)
    }
}
```

Izlaz

```
subject1 is always equal to subject2
subject1 is always equal to subject2
subject1 is always equal to subject2
```

Jediničnolika struktura bez polja: AlwaysEqual u memoriji



Slika 4: AlwaysEqual na stack-u

Vlasništvo u strukturama

- Rust strukture obično koriste tipove sa vlasništvom kao što je String.
  - Umesto pozajmljenih tipova kao što je &str.
- Ovaj pristup obezbeđuje da svaka instanca strukture poseduje svoje podatke.
  - Podaci ostanu važeći dokle god je i struktura važeća.
- Strukture mogu čuvati reference na podatke koji su u vlasništvu nekog drugog.
  - Za to su potrebni eksplicitni životni vekovi (lifetimes).
- Životni vekovi obezbeđuju da podaci na koje struktura referiše ostanu važeći koliko i sama struktura.

Vlasništvo u strukturama: primer

primeri/structs-enums/src/bin/nocompile_struct_string_slices.rs

```
// This does not compile, because the fields of the struct are references.
// Resolve this by using the `String` type instead of `&str`
// Or by using the `lifetime` syntax.
struct User {
    active: bool,
    username: &str,
```

```
    email: &str,
    sign_in_count: u64,
}

fn main() {
    let user1 = User {
        active: true,
        username: "someusername123",
        email: "someone@example.com",
        sign_in_count: 1,
    };
}
```

Izlaz

```
error[E0106]: missing lifetime specifier
 --> structs-enums/src/bin/nocompile_struct_string_slices.rs:6:15
  |
6 |     username: &str,
  |               ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
4 ~ struct User<'a> {
5 |     active: bool,
6 ~     username: &'a str,
  |

error[E0106]: missing lifetime specifier
 --> structs-enums/src/bin/nocompile_struct_string_slices.rs:7:12
  |
7 |     email: &str,
  |            ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
4 ~ struct User<'a> {
5 |     active: bool,
6 |     username: &str,
7 ~     email: &'a str,
  |

For more information about this error, try `rustc --explain E0106`.
error: could not compile `structs-enums` (bin
 ↪  "nocompile_struct_string_slices") due to 2 previous errors
```

Pozajmljivanje polja u strukturama

- Vodi se računa o vlasništvu i pozajmljivanju na nivou:
  - Strukture,
  - Polja.
- Ukoliko se pozajmi polje strukture:
  - Suštinski je pozajmljeno:
    - Polje,
    - Sama struktura, ali ne i njena ostala polja.

Pozajmljivanje polja u strukturama: primer

primeri/structs-enums/src/bin/struct_field_borrowing.rs

```rust
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let mut p = Point { x: 0, y: 0 }; // `p` has full access here

    let x = &mut p.x; // Mutably borrowing `p.x`

    // At this point:
    // - `p.x` is exclusively borrowed mutably, so only `x` can access it.
    // - `p` is partially restricted: cannot be accessed immutably because
     ↪  of the mutable borrow on `p.x`.
    // - `p.y` is unaffected; we can still access it independently.

    // This will not work:
    // let p_ref = &p;  // Immutable borrow on `p`
    // println!("p.x is not accessible: {}", p.x);  // `p.x` is not
     ↪  accessible

    // We can access `p.y` without issues:
    println!("p.y is still accessible: {}", p.y); // `p.y` can be accessed

    // Modify `p.x` through `x`:
    *x += 1; // This works because `x` is a mutable reference to `p.x`

    println!("Updated p.x = {}", p.x); // `p.x` can be read after
     ↪  modifying
}
```

Izlaz

```
p.y is still accessible: 0
Updated p.x = 1
```

Metode i asocirane funkcije

- Metode se definišu u `impl` bloku strukture (ili enumeracije).
- Prvi parametar metode je uvek `self`, instanca nad kojom se metoda poziva.
  - `Self` je alias za tip iz `impl` bloka.
- `self` se može uzeti na tri načina:
  - `&self`: nepromenljivo pozajmljivanje,
  - `&mut self`: promenljivo pozajmljivanje, kad metoda menja instancu,
  - `self`: preuzimanje vlasništva; retko, najčešće kad metoda pretvara instancu u nešto drugo.
- Asocirana funkcija nema `self` i poziva se sa `::` (`Account::new`); često služi kao konstruktor.
- Pri pozivu `account.deposit(100)` Rust sam dodaje `&`, `&mut` ili `*` (automatsko referenciranje).

Metode: self, &self, &mut self i asocirana funkcija

primeri/structs-enums/src/bin/methods.rs

```rust
struct Account {
    owner: String,
    balance: u64,
}

impl Account {
    // Associated function (no `self`), called as `Account::new(...)`
    fn new(owner: String) -> Self {
        Account { owner, balance: 0 }
    }

    // `&self` borrows the instance immutably
    fn balance(&self) -> u64 {
        self.balance
    }

    // `&mut self` borrows the instance mutably
    fn deposit(&mut self, amount: u64) {
        self.balance += amount;
    }

    // `self` takes ownership: `account` is unusable afterwards
    fn close(self) -> u64 {
        println!("Closing the account of {}", self.owner);
        self.balance
    }
}

fn main() {
    let mut account = Account::new(String::from("Alice"));
    account.deposit(100);
    println!("Balance: {}", account.balance());

    let paid_out = account.close();
    println!("Paid out: {}", paid_out);
    // println!("{}", account.balance()); // Error: `account` was moved
}
```

Izlaz

```
Balance: 100
Closing the account of Alice
Paid out: 100
```

Enumeracije

Enumeracije

- Ukoliko se vrednost promenljive uzima iz nekog konačnog skupa vrednosti:
  - Koji nije nepraktično nabrojati.
    - Koristiti enumeracije.

Enumeracije: primer

primeri/structs-enums/src/bin/enum.rs

```rust
// This enumeration is used to define the type of IP address.
enum IpAddrKind {
```

```
    V4,
    V6,
}

// This struct is used to define the IP address.
struct IpAddr {
    kind: IpAddrKind,
    address: String,
}
fn main() {
    let home = IpAddr {
        kind: IpAddrKind::V4,
        address: String::from("127.0.0.1"),
    };

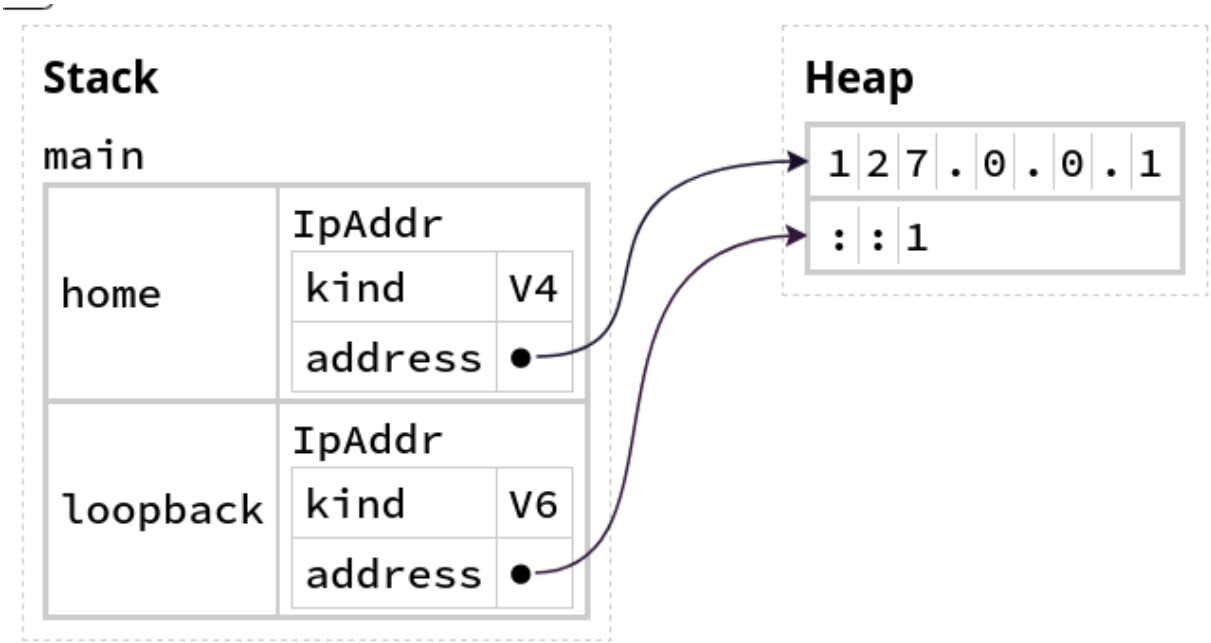
    let loopback = IpAddr {
        kind: IpAddrKind::V6,
        address: String::from("::1"),
    };

    match home.kind {
        IpAddrKind::V4 => println!("Home address is: V4"),
        IpAddrKind::V6 => println!("Home address is: V6"),
    }
}
```

Izlaz

```
Home address is: V4
```

Enumeracije: IpAddr u memoriji



Slika 5: IpAddr: V4 i V6 na stack-u i heap-u

Enumeracije: varijante sa podacima

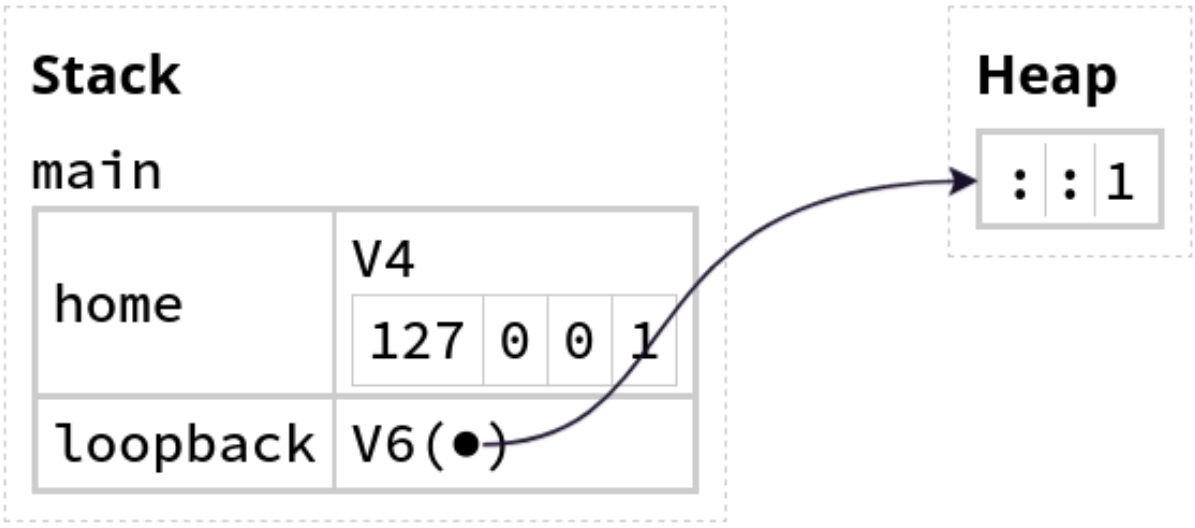
primeri/structs-enums/src/bin/enum_tuple.rs

```rust
#[derive(Debug, Clone)]
enum IpAddr {
    V4(u8, u8, u8, u8), // Tuple struct
    V6(String),         // Tuple struct
}
fn main() {
    let home = IpAddr::V4(127, 0, 0, 1);
    let loopback = IpAddr::V6(String::from("::1"));
    let home2 = home.clone();
    println!("{:?}", home);
}
```

Izlaz

```
V4(127, 0, 0, 1)
```

Enumeracije: varijante sa podacima u memoriji



Slika 6: Enumeracija sa podacima u varijantama

Enumeracije: osobina Copy

primeri/structs-enums/src/bin/enum_copy.rs

```rust
// Derives the Copy, Clone, and Debug traits for the enum automatically.
// This works because:
// 1. `Direction` has no data that isn't `Copy` (e.g., references,
 ↪  `String`, or `Vec`).
// 2. All enum variants are simple and contain no additional fields.
#[derive(Copy, Clone, Debug)]
enum Direction {
    North,
    South,
    East,
    West,
}
```

```
fn main() {
    let direction = Direction::North; // `direction` is created
    let copy_of_direction = direction; // `direction` is copied to
     ↪  `copy_of_direction`

    println!("{:?}", direction); // Outputs: North
    println!("{:?}", copy_of_direction); // Outputs: North
}
```

Izlaz

```
North
North
```

Enumeracije: metoda `call` i više tipova podataka

primeri/structs-enums/src/bin/enum_multiple.rs

```
// Enums can have multiple types of data
enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

// Implement a method for the enum
impl Message {
    fn call(&self) {
        match self {
            Message::Quit => println!("Quit"),
            Message::Move { x, y } => println!("Move to x: {}, y: {}", x,
             ↪  y),
            Message::Write(s) => println!("Write: {}", s),
            Message::ChangeColor(r, g, b) => {
                println!("Change color to r: {}, g: {}, b: {}", r, g, b)
            }
        }
    }
}
fn main() {
    let m = Message::Write(String::from("hello"));
    m.call();
    m.call();

    let m = Message::Move { x: 1, y: 2 };
    m.call();
    m.call();

    let m = Message::ChangeColor(255, 0, 0);
    m.call();
    m.call();

    let m = Message::Quit;
    m.call();
    m.call();
}
```

Izlaz

```
Write: hello
Write: hello
Move to x: 1, y: 2
Move to x: 1, y: 2
Change color to r: 255, g: 0, b: 0
Change color to r: 255, g: 0, b: 0
Quit
Quit
```

`Option<T>`: odsustvo vrednosti

- Rust nema `null`; odsustvo vrednosti se izražava enumeracijom `Option<T>`, definisanom u standardnoj biblioteci:

```rust
enum Option<T> {
    None,
    Some(T),
}
```

- `Some` i `None` se mogu koristiti bez prefiksa `Option::`.
- `Option<T>` i `T` su različiti tipovi: pre upotrebe vrednosti mora se obraditi i slučaj `None` (na primer, pomoću `match`).

`Option<T>`: sabiranje sa brojem (greška)

primeri/structs-enums/src/bin/nocompile_option_not_t.rs

```rust
fn main() {
    let x: i8 = 5;
    let y: Option<i8> = Some(5);

    let sum = x + y; // Error: cannot add `Option<i8>` to `i8`
    println!("{}", sum);
}
```

Izlaz

```
error[E0277]: cannot add `Option<i8>` to `i8`
 --> structs-enums/src/bin/nocompile_option_not_t.rs:5:17
  |
5 |     let sum = x + y; // Error: cannot add `Option<i8>` to `i8`
  |                 ^ no implementation for `i8 + Option<i8>`
  |
  = help: the trait `Add<Option<i8>>` is not implemented for `i8`
help: the following other types implement trait `Add<Rhs>`
 --> /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/
 ↪  ops/arith.rs:98:8
  |
  = note: `i8` implements `Add`
 ::: /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/
 ↪  ops/arith.rs:113:0
  |
  = note: in this macro invocation
 --> /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/
 ↪  internal_macros.rs:22:8
  |
  = note: `&i8` implements `Add<i8>`
```

```
 ::: /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/
 ↪  internal_macros.rs:33:8
  |
  = note: `i8` implements `Add<&i8>`
 ::: /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/
 ↪  internal_macros.rs:44:8
  |
  = note: `&i8` implements `Add`
  = note: this error originates in the macro `add_impl` (in Nightly
 ↪  builds, run with -Z macro-backtrace for more info)

For more information about this error, try `rustc --explain E0277`.
error: could not compile `structs-enums` (bin "nocompile_option_not_t")
 ↪  due to 1 previous error
```

Generički tipovi

Generički tipovi u definicijama funkcija

- Generički tipovi omogućavaju fleksibilnost u parametrima i povratnim vrednostima.
  - Smanjuju potrebu za ponavljanjem koda.
- Generički tipovi se dodaju unutar uglastih zagrada <T> posle imena funkcije.
  - T se koristi kao oznaka za tip.
- Omogućava definisanje funkcija koje mogu da rade sa bilo kojim tipom T.
- Da bi se omogućile određene operacije, kao što su poređenja, mogu se dodati osobine (traits).
  - PartialOrd.

Generički tipovi: dupliran kod bez generika

primeri/structs-enums/src/bin/generic_intro.rs

```rust
// Function for finding the largest number in a list of i32
fn largest_i32(list: &[i32]) -> &i32 {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

// Function for finding the largest char in a list of char
fn largest_char(list: &[char]) -> &char {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}
```

```
// This is actually how monomorphization works in Rust
// The compiler generates code for each type that is used in the generic
 ↪  function
// This benefits from the performance of static typing and the flexibility
 ↪  of generics
fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let result = largest_i32(&number_list);
    println!("The largest number is {result}");

    let char_list = vec!['y', 'm', 'a', 'q'];

    let result = largest_char(&char_list);
    println!("The largest char is {result}");
}
```

Izlaz

```
The largest number is 100
The largest char is y
```

Generički tipovi: generička funkcija largest

primeri/structs-enums/src/bin/generic_func.rs

```
// This is a generic function that takes a slice of any type that
 ↪  implements
// the PartialOrd and returns a reference to the largest item in the
 ↪  slice.
fn largest<T: std::cmp::PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

// Chars and i32s are both types that implement the PartialOrd trait
fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let result = largest(&number_list);
    println!("The largest number is {result}");

    let char_list = vec!['y', 'm', 'a', 'q'];

    let result = largest(&char_list);
    println!("The largest char is {result}");
}
```

Izlaz

```
The largest number is 100
The largest char is y
```

Generički tipovi u definicijama struktura i metoda

- Moguće je napraviti strukturu koja u sebi sadrži proizvoljan tip.
- Sintaksa `StructureName<T>`.

Generički tipovi: struktura Point sa jednim tipom

primeri/structs-enums/src/bin/generic_struct.rs

```rust
// Structure with generic type for x and y
// Both x and y must be of the same type
// The type of x and y is defined when the structure is created
#[derive(Debug)]
struct Point<T> {
    x: T,
    y: T,
}

// Implementation of the structure
impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

fn main() {
    let integer = Point { x: 5, y: 10 };
    let float = Point { x: 1.0, y: 4.0 };
    println!("{:?}", integer);
    println!("{:?}", float);

    println!("x of integer: {}", integer.x());
    println!("x of float: {}", float.x());
}
```

Izlaz

```
Point { x: 5, y: 10 }
Point { x: 1.0, y: 4.0 }
x of integer: 5
x of float: 1
```

Generički tipovi: Point sa dva tipa

primeri/structs-enums/src/bin/generic_struct_two_types.rs

```rust
// x and y can be of different types
// Although, now structure signature has two generic types T and U
// The type of x and y is defined when the structure is created
#[derive(Debug)]
struct Point<T, U> {
    x: T,
    y: U,
}
```

```rust
fn main() {
    let both_integer = Point { x: 5, y: 10 };
    let both_float = Point { x: 1.0, y: 4.0 };
    let integer_and_float = Point { x: 5, y: 4.0 };

    println!("{:?}", both_integer);
    println!("{:?}", both_float);
    println!("{:?}", integer_and_float);
}
```

Izlaz

```
Point { x: 5, y: 10 }
Point { x: 1.0, y: 4.0 }
Point { x: 5, y: 4.0 }
```

Generički tipovi: metoda mixup

primeri/structs-enums/src/bin/generic_struct_mixup.rs

```rust
// Structure that allows for different types for x and y
struct Point<X1, Y1> {
    x: X1,
    y: Y1,
}

impl<X1, Y1> Point<X1, Y1> {
    // Method that allows for yet another different types for x and y
    fn mixup<X2, Y2>(self, other: Point<X2, Y2>) -> Point<X1, Y2> {
        Point {
            x: self.x,
            y: other.y,
        }
    }
}

fn main() {
    // This means that three different implementations of
    // the structure Point are created (Monomorphization)
    let p1 = Point { x: 5, y: 10.4 };
    let p2 = Point { x: "Hello", y: 'c' };
    let p3 = p1.mixup(p2);

    println!("p3.x = {}, p3.y = {}", p3.x, p3.y);
}
```

Izlaz

```
p3.x = 5, p3.y = c
```

Generički tipovi u enumeracijama

- Enumeracija može imati generičke parametre: tip podatka u varijanti je parametar.
- Neke od dominantnih enumeracija:

```rust
enum Option<T> {
    Some(T),
    None,
```

```
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

- `Result<T, E>` se koristi za greške (`Ok` za uspeh, `Err` za grešku); detaljnije na predavanju o rukovanju greškama.

Generički tipovi u enumeracijama: primer

`primeri/structs-enums/src/bin/generic_enums.rs`

```
// &Vec<&str> is automatically converted to &[&str] (deref coercion)
fn find_word(words: &[&str], target: &str) -> Option<usize> {
    // Loop through the words to find the target
    for (index, &word) in words.iter().enumerate() {
        if word == target {
            return Some(index);
        }
    }
    None
}

fn divide_numbers(dividend: f64, divisor: f64) -> Result<f64, String> {
    // Check if division is possible (divisor not zero)
    if divisor == 0.0 {
        Err("Cannot divide by zero.".to_string())
    } else {
        Ok(dividend / divisor)
    }
}

fn main() {
    // Using Option to handle optional values
    let words = vec!["apple", "banana", "cherry"];
    let target = "banana";
    let result = find_word(&words, target);
    match result {
        Some(index) => println!("Found '{}' at index: {}", target, index),
        None => println!("'{}' not found", target),
    }

    let missing_word = "grape";
    let result = find_word(&words, missing_word);
    match result {
        Some(index) => println!("Found '{}' at index: {}", missing_word,
        ↪  index),
        None => println!("'{}' not found", missing_word),
    }

    // Using Result to handle potential errors
    let result = divide_numbers(10.0, 2.0);
    match result {
        Ok(result) => println!("10.0 divided by 2.0 is: {}", result),
        Err(e) => println!("Error: {}", e),
    }
```

```
    let result = divide_numbers(10.0, 0.0);
    match result {
        Ok(result) => println!("10.0 divided by 0.0 is: {}", result),
        Err(e) => println!("Error: {}", e),
    }
}
```

Izlaz

```
Found 'banana' at index: 1
'grape' not found
10.0 divided by 2.0 is: 5
Error: Cannot divide by zero.
```

Performanse generičkih tipova?

- Monomorfizacija (*monomorphization*):
  - Kompajler za svaki konkretan tip koji se koristi generiše zasebnu implementaciju.

```
enum Option_i32 {
    Some(i32),
    None,
}

enum Option_f64 {
    Some(f64),
    None,
}

fn main() {
    let integer = Option_i32::Some(5);
    let float = Option_f64::Some(5.0);
}
```

Pattern matching

match: enumeracije i Option

- Šablon može biti varijanta enumeracije, uključujući `Some(i)` i `None` za `Option<T>`.
- Šablon vezuje podatke iz varijante: `Some(i)` vezuje vrednost koja se nalazi u `Some`.
- Kompajler proverava da je pokrivena svaka varijanta, grana koja nedostaje je greška pri kompajliranju.

match: iscrpnost grana

primeri/structs-enums/src/bin/match_exhaustive.rs

```
// match is exhaustive: every variant of Option must be handled
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        Some(i) => Some(i + 1),
        None => None,
    }
}

fn main() {
```

```
    let five = Some(5);
    let six = plus_one(five);
    let none = plus_one(None);
    println!("{:?}", five);
    println!("{:?}", six);
    println!("{:?}", none);
}
```

Izlaz

```
Some(5)
Some(6)
None
```

match: iscrpnost grana (greška)

primeri/structs-enums/src/bin/nocompile_match_exhaustive.rs

```
// This will not compile: the match is not exhaustive
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        Some(i) => Some(i + 1),
    }
}

fn main() {
    let six = plus_one(Some(5));
    println!("{:?}", six);
}
```

Izlaz

```
error[E0004]: non-exhaustive patterns: `None` not covered
 --> structs-enums/src/bin/nocompile_match_exhaustive.rs:3:11
  |
3 |     match x {
  |           ^ pattern `None` not covered
  |
note: `Option<i32>` defined here
 --> /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/
 ↪  option.rs:598:0
 ::: /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/
 ↪  option.rs:602:4
  |
  = note: not covered
  = note: the matched value is of type `Option<i32>`
help: ensure that all possible cases are being handled by adding a match
 ↪  arm with a wildcard pattern or an explicit pattern as shown
  |
4 ~         Some(i) => Some(i + 1),
5 ~         None => todo!(),
  |

For more information about this error, try `rustc --explain E0004`.
error: could not compile `structs-enums` (bin
 ↪  "nocompile_match_exhaustive") due to 1 previous error
```

match: pozivi funkcija u granama

primeri/structs-enums/src/bin/match_arms.rs

```rust
fn main() {
    let dice_roll = 7;
    // Within match, functions can be called
    match dice_roll {
        3 => add_fancy_hat(),
        7 => remove_fancy_hat(),
        _ => (),
    }

    fn add_fancy_hat() {
        println!("Added a fancy hat");
    }
    fn remove_fancy_hat() {
        println!("Removed a fancy hat");
    }
}
```

Izlaz

```
Removed a fancy hat
```

match: vezivanje premešta vrednost (greška)

primeri/structs-enums/src/bin/nocompile_match_moves.rs

```rust
fn main() {
    let opt: Option<String> = Some(String::from("Hello world"));

    match opt {
        Some(value) => println!("{}", value), // `value` takes the String
         ↪  out of `opt`
        None => println!("None!"),
    };

    println!("{:?}", opt); // Error: `opt` is partially moved
}
```

Izlaz

```
error[E0382]: borrow of partially moved value: `opt`
 --> structs-enums/src/bin/nocompile_match_moves.rs:9:22
  |
5 |         Some(value) => println!("{}", value), // `value` takes the
 ↪  String out of `opt`
  |              ----- value partially moved here
...
9 |     println!("{:?}", opt); // Error: `opt` is partially moved
  |                      ^^^ value borrowed here after partial move
  |
  = note: partial move occurs because value has type `String`, which does
 ↪  not implement the `Copy` trait
help: borrow this binding in the pattern to avoid moving the value
  |
5 |         Some(ref value) => println!("{}", value), // `value` takes the
 ↪  String out of `opt`
  |              +++
```

```
For more information about this error, try `rustc --explain E0382`.
error: could not compile `structs-enums` (bin "nocompile_match_moves") due
 ↪  to 1 previous error
```

match: vlasništvo nad vrednostima

primeri/structs-enums/src/bin/pattern_matching_ownership.rs

```rust
fn demonstrate_placeholder() {
    let opt: Option<String> = Some(String::from("Hello world"));

    match opt {
        Some(_) => println!("Some!"), // `_` does not bind, nothing moved
        None => println!("None!"),
    };

    match opt {
        Some(_) => println!("Some!"), // `opt` is still usable
        None => println!("None!"),
    };

    println!("{:?}", opt);
}

fn demonstrate_borrowing_1() {
    let opt: Option<String> = Some(String::from("Hello world"));

    // Matching a reference binds `value` as `&String`
    // (default binding mode)
    match &opt {
        Some(value) => println!("{}", value), // borrowed, not moved
        None => println!("None!"),
    };

    match &opt {
        Some(value) => println!("{}", value), // `opt` is still usable
        None => println!("None!"),
    };

    println!("{:?}", opt);
}

fn demonstrate_borrowing_2() {
    let opt: Option<String> = Some(String::from("Hello world"));

    match opt {
        Some(ref value) => println!("{}", value), // `ref` borrows
        None => println!("None!"),
    };

    match opt {
        Some(ref value) => println!("{}", value), // `opt` is still usable
        None => println!("None!"),
    };

    println!("{:?}", opt);
}
```

```
fn main() {
    demonstrate_placeholder();
    demonstrate_borrowing_1();
    demonstrate_borrowing_2();
}
```

Izlaz

```
Some!
Some!
Some("Hello world")
Hello world
Hello world
Some("Hello world")
Hello world
Hello world
Some("Hello world")
```

match: `&` ili `ref`

- Podrazumevano se poklapa referenca (`match &x`): vrednosti se vezuju po referenci (`&T`), a `x` ostaje čitav.
- `ref` je potreban samo kada isti šablon jedno polje premešta, a drugo pozajmljuje.
- Posle takvog delimičnog premeštanja ne sme se koristiti celina, ali se preostala polja mogu koristiti.
- Tipovi koji implementiraju `Drop` ne dozvoljavaju delimično premeštanje.

match: delimično premeštanje

primeri/structs-enums/src/bin/partial_move.rs

```
struct User {
    name: String,
    email: Option<String>,
}

fn main() {
    let user = User {
        name: String::from("Ana"),
        email: Some(String::from("ana@example.com")),
    };

    // `name` is moved out of `user`, `email` is only borrowed
    let User { name, ref email } = user;
    println!("{} {:?}", name, email);

    // Only `user.name` is moved: other fields are still usable
    println!("{:?}", user.email);
    // println!("{}", user.name); // Error: `user.name` was moved
}
```

Izlaz

```
Ana Some("ana@example.com")
Some("ana@example.com")
```

match: celina posle delimičnog premeštanja (greška)

primeri/structs-enums/src/bin/nocompile_partial_move_whole.rs

```rust
struct User {
    name: String,
    email: Option<String>,
}

fn main() {
    let user = User {
        name: String::from("Ana"),
        email: Some(String::from("ana@example.com")),
    };

    let User { name, ref email } = user;
    println!("{} {:?}", name, email);

    let _r = &user; // Error: `user` is partially moved
}
```

Izlaz

```
error[E0382]: borrow of partially moved value: `user`
  --> structs-enums/src/bin/nocompile_partial_move_whole.rs:15:14
   |
12 |     let User { name, ref email } = user;
   |                ---- value partially moved here
...
15 |     let _r = &user; // Error: `user` is partially moved
   |              ^^^^^ value borrowed here after partial move
   |
   = note: partial move occurs because `user.name` has type `String`,
 ↪  which does not implement the `Copy` trait
help: borrow this binding in the pattern to avoid moving the value
   |
12 |     let User { ref name, ref email } = user;
   |                +++

For more information about this error, try `rustc --explain E0382`.
error: could not compile `structs-enums` (bin
 ↪  "nocompile_partial_move_whole") due to 1 previous error
```

Drop i delimično premeštanje

- Osobina `Drop` ima metodu `drop(&mut self)`, koju Rust poziva automatski kada vlasnik izađe iz opsega.
- Metoda `drop` dobija celu vrednost i sme da koristi sva njena polja.
- Kada bi se polje premestilo iz takve vrednosti, `drop` bi pristupio premeštenom polju, pa kompajler zabranjuje premeštanje polja iz tipa koji implementira `Drop`.
- Polje se umesto toga pozajmljuje (`&conn.host`) ili kopira (`.clone()`).

match: delimično premeštanje i Drop (greška)

primeri/structs-enums/src/bin/nocompile_partial_move_drop.rs

```rust
struct Connection {
    host: String,
}
```

```
impl Drop for Connection {
    fn drop(&mut self) {
        println!("closing {}", self.host);
    }
}

fn main() {
    let conn = Connection { host: String::from("localhost") };

    let Connection { host } = conn; // Error: `Connection` implements
     ↪  `Drop`
    println!("{}", host);
}
```

Izlaz

```
error[E0509]: cannot move out of type `Connection`, which implements the
 ↪  `Drop` trait
  --> structs-enums/src/bin/nocompile_partial_move_drop.rs:14:31
   |
14 |     let Connection { host } = conn; // Error: `Connection` implements
 ↪  `Drop`
   |                      ----     ^^^^ cannot move out of here
   |                      |
   |                      data moved here because `host` has type
 ↪  `String`, which does not implement the `Copy` trait
   |
help: consider borrowing the pattern binding
   |
14 |     let Connection { ref host } = conn; // Error: `Connection`
 ↪  implements `Drop`
   |                      +++

For more information about this error, try `rustc --explain E0509`.
error: could not compile `structs-enums` (bin
 ↪  "nocompile_partial_move_drop") due to 1 previous error
```

match: pozajmljivanje svih polja

primeri/structs-enums/src/bin/borrow_all_fields.rs

```
struct User {
    name: String,
    email: Option<String>,
}

fn main() {
    let user = User {
        name: String::from("Ana"),
        email: Some(String::from("ana@example.com")),
    };

    // Matching a reference: every field is borrowed, nothing is moved
    let User { name, email } = &user;
    println!("{} {:?}", name, email); // `name`: &String, `email`:
     ↪  &Option<String>
```

```
    // `user` is still whole and usable
    let r = &user;
    println!("{} {:?}", r.name, r.email);
}
```

Izlaz

```
Ana Some("ana@example.com")
Ana Some("ana@example.com")
```

if let: skraćeni match

primeri/structs-enums/src/bin/if_let.rs

```
#[derive(Debug)]
enum UsState {
    Alabama,
    Alaska,
    // --snip--
}

enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}

fn main() {
    let penny = Coin::Penny;
    let quarter = Coin::Quarter(UsState::Alaska);
    let mut count = 0;

    match &penny {
        Coin::Quarter(state) => println!("State quarter from {state:?}!"),
        _ => count += 1,
    }

    if let Coin::Quarter(state) = &penny {
        println!("State quarter from {state:?}!");
    } else {
        count += 1;
    }

    if let Coin::Quarter(state) = penny {
        println!("State quarter from {state:?}!");
    }

    if let Coin::Quarter(state) = quarter {
        if let UsState::Alaska = state {
            println!("State quarter from Alaska!");
        } else {
            count += 1;
        }
    } else {
        count += 1;
    }
    // println!("Quarter: {:?}", quarter);
```

```
    println!("Count: {}", count);
}
```

Izlaz

```
State quarter from Alaska!
Count: 2
```

Zadatak

Zadatak 1

Napisati program u Rust programskom jeziku sa strukturama `Point` (polja `x` i `y` tipa `f32`) i `Rectangle` (polja `top_left` i `bottom_right` tipa `Point`), a zatim:

- Napisati funkciju `rect_area` koja računa površinu pravougaonika.
- Napisati funkciju `square` koja prima `Point` i `f32`, a vraća `Rectangle` čiji je gornji levi ugao u zadatoj tački, a širina i visina su jednake zadatoj vrednosti.
- Napraviti pravougaonik u `main` funkciji i ispisati površinu pravougaonika i kvadrata.

Zadatak 2

Napisati program u Rust programskom jeziku sa enumeracijom `Message` koja ima varijante:

- `Resize { width, height }`, gde su širina i visina tipa `u64`.
- `Move(Point)`, gde je `Point` struktura sa poljima `x` i `y` tipa `u64`.
- `Echo(String)`.
- `ChangeColor(u8, u8, u8)`.
- `Quit`.

Zatim napraviti strukturu `State` sa poljima `width`, `height`, `position`, `message`, `color` (torka `(u8, u8, u8)`) i `quit` (`bool`), i implementirati metodu `process(&mut self, message: Message)` koja pomoću `match` izraza menja odgovarajuća polja. Testirati program slanjem po jedne poruke svake varijante.

Izvori

Izvori

- Rust Community. “The Rust Programming Language - the Rust Programming Language.” Rust-Lang.org, 2018, doc.rust-lang.org/book/.
- Crichton, Will. “Experiment Introduction - the Rust Programming Language.” Brown.edu, rust-book.cs.brown.edu/.
- Rust Community. “Tour of Rust - Let’s Go on an Adventure!” Tourofrust.com, tourofrust.com/.
- Rust Team. “Rust Programming Language.” Rust-Lang.org, 2018, www.rust-lang.org/.
- Rust Community. “Structures - Rust by Example.” Rust-Lang.org, doc.rust-lang.org/rust-by-example/custom_types/structs.html.
- Rust Community. “Patterns - the Rust Reference.” Rust-Lang.org, doc.rust-lang.org/reference/patterns.html.
- Rust Community. “The ref Pattern - Rust by Example.” Rust-Lang.org, doc.rust-lang.org/rust-by-example/scope/borrow/ref.html.
- Rust Community. “Rustlings.” GitHub, github.com/rust-lang/rustlings (vežbe `08_enums`: `enums2.rs`, `enums3.rs`).